SOFTWARE REQUIREMENT SPECIFICATION

HACKAGON

A Web Platform for Hosting and Joining Online Coding Competitions

Version 1.0

Date: 11 October 2026

Prepared by

Hiten Sharma

Harsh Sharma

Hemanth Singh Tanwar

Arjun Suman

Akshat Rathore

Department of Computer Science and Engineering

Arya College of Engineering and IT, Jaipur

DOCUMENT CONTROL

Revision History

Version	Date	Description
1.0	11 Oct 2026	First complete specification matching the current build

Contents of This Document

The table of contents follows on the next page. Each major section starts on its own page.

TABLE OF CONTENTS

1. INTRODUCTION.....4
1.1 Purpose.....4
1.2 Scope.....4
1.3 Definitions and Acronyms.....4
1.4 References.....4
1.5 Overview of this Document.....5
2. OVERALL DESCRIPTION.....6
2.1 Product Perspective.....6
2.2 Product Functions.....6
2.3 User Classes.....6
2.4 Operating Environment.....6
2.5 Design and Implementation Constraints.....6
2.6 Assumptions and Dependencies.....7
3. FUNCTIONAL REQUIREMENTS.....8
3.1 Accounts and Verification.....8
3.2 Rooms and Matches.....8
3.3 Judging and Question Bank.....9
3.4 Tournaments and Money.....9
3.5 Proctoring.....10
3.6 Learning and Achievements.....10
3.7 Administration.....10
4. EXTERNAL INTERFACE REQUIREMENTS.....12
4.1 User Interfaces.....12
4.2 Software Interfaces.....12
4.3 Communication Interfaces.....12
5. NON-FUNCTIONAL REQUIREMENTS.....13
6. DATA REQUIREMENTS.....14
6.1 Data Model.....14
6.2 Data Rules.....14
7. SYSTEM MODELS.....15
7.1 Sign-up and OTP Flow.....15
7.2 Match, Judging and Proctoring Flow.....15
7.3 Tournament Money Flow.....16
8. BUSINESS RULES AND REQUIREMENT STATUS.....17
8.1 Business Rules.....17
8.2 Status Summary.....17
8.3 Future Requirements.....17
APPENDIX A: API ENDPOINTS.....18
APPENDIX B: TRACEABILITY.....19

1. INTRODUCTION

1.1 Purpose

This Software Requirement Specification (SRS) describes the requirements of HackAgon, a web application for hosting and joining online coding competitions. It is written for the project team, the project guide and anyone who will test, maintain or extend the system. It states what the system must do, the limits it works within, and which requirements are met by the current build, which are simulated and which are planned.

1.2 Scope

HackAgon lets any verified user host a contest in solo, duo, squad or duel format, lets others join it, judges submissions automatically, ranks players, and optionally runs paid tournaments in which the prize pool is funded in advance, entry fees are held in escrow, and prizes are paid by a published split after a flat 1% platform fee on the prize pool. A proctoring feature (camera gate, webcam snapshots and a server-side tab-lock) protects fairness. A learning area (mistake book, free library, achievements, AI coach) helps players improve. An administrator back office manages questions, books, users and disputes.

Out of scope for this version: native mobile apps, team chat, video calls, live spectating, and real-money payment processing.

1.3 Definitions and Acronyms

Term	Meaning
Room	A contest that a host creates; it can be a casual room, a duel or a tournament
Tournament	A room with a prize pool and optionally an entry fee
Host	The user who creates and runs a room
Escrow	Money held by the platform until the result is final
Paise	One hundredth of a rupee; all money is stored as integer paise
OTP	One-time password, a 6-digit code sent to the user
JWT	JSON Web Token used to keep a user signed in
Tab-lock	Server-side lock applied to a player after a proctoring violation
Hidden test case	A test case the player never sees, used only by the judge
API	The JSON interface the pages use to talk to the server, under /api

1.4 References

- Java SE 17 documentation (docs.oracle.com/en/java/javase/17).
- Spring Boot 3.2 reference documentation (docs.spring.io/spring-boot).
- MySQL 8.0 Reference Manual and TiDB Cloud documentation.
- RFC 7519, JSON Web Token (JWT), IETF, 2015.
- Provos and Mazieres, A Future-Adaptable Password Scheme, USENIX, 1999.
- HackAgon source repository, github.com/ASR-C/HACK-AGON.

1.5 Overview of this Document

Section 2 gives the overall description. Section 3 lists the functional requirements by feature. Section 4 covers the external interfaces. Section 5 gives non-functional requirements. Section 6 describes the data. Section 7 shows system models. Section 8 states business rules and the status of each requirement area. The appendices list the API endpoints and trace requirements to modules and tests.

2. OVERALL DESCRIPTION

2.1 Product Perspective

HackAgon is a self-contained web application. The browser loads fifteen HTML pages from a Spring Boot server and calls a JSON API. The server stores data in a MySQL-compatible database (TiDB Cloud in production) and uses two outside services: an e-mail service for OTPs and the Groq API for the AI coach.

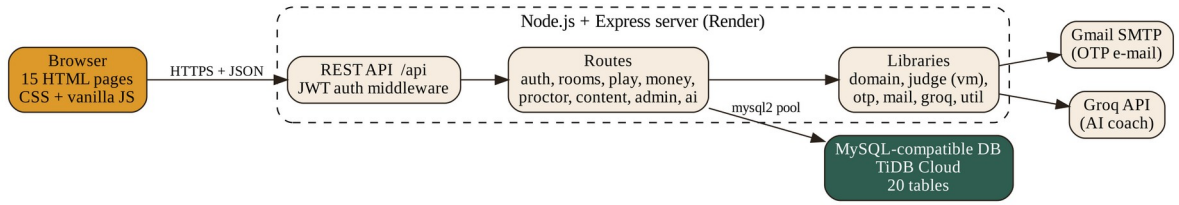


Figure 2.1: System architecture

2.2 Product Functions

- Account sign-up, e-mail OTP verification, login and password reset
- Creating, listing, joining and running rooms in four formats
- Judging submissions against hidden test cases, scoring and ranking
- Paid tournaments with escrow, prize presets, 1% fee, refunds and a ledger
- Proctoring: camera gate, snapshots, tab-lock, host unlock and eliminate
- Learning: mistake book, free library, achievements, AI coach
- Administration: question bank, library review, users, disputes, money totals

2.3 User Classes

User class	Description and main needs
Visitor	Not signed in; reads the landing page, signs up or logs in
Player	Signed-in user who joins rooms, solves questions, reviews mistakes and reads library books
Host	A player who creates rooms and tournaments, funds prize pools and manages a match
Administrator	Manages the question bank, library, users and disputes; sees money totals

2.4 Operating Environment

Server: Java 17 (JDK 17 or newer, built with Maven) on Render (Linux). Database: MySQL 8 compatible service. Client: current desktop Chrome, Edge or Firefox for matches, with a working webcam for proctored rooms; phones and tablets are supported for browsing.

2.5 Design and Implementation Constraints

- The system uses HTML, CSS and vanilla JavaScript on the client, and Java 17 with Spring Boot on the server.
- All money is held as integer paise; floating point is not used for money.
- All secrets are read from environment variables and never stored in the repository.

- The free hosting tier sleeps after inactivity, loses local files on restart and blocks outbound SMTP ports; e-mail therefore goes through an HTTPS e-mail API on Render.
- Contests must be skill-only; no draws, spins or random prizes.

2.6 Assumptions and Dependencies

- Players have a stable internet connection and a modern browser.
- An e-mail service (Gmail SMTP or Brevo) and the Groq API are reachable from the server.
- A payment gateway and SMS provider will be integrated before real money or SMS OTP is used.
- Hosts and players accept the rules page, including the limits of browser-based proctoring.

3. FUNCTIONAL REQUIREMENTS

Requirement status: Implemented means it works in the current build; Simulated means the behaviour exists but a stand-in replaces the real service; Planned means it is not yet built.

3.1 Accounts and Verification

ID	Requirement	Status
AUTH-1	The system shall let a visitor sign up with name, username (3 to 20 letters, digits or underscore), e-mail, 10-digit mobile, password of at least 8 characters, and date of birth.	Implemented
AUTH-2	The system shall reject a duplicate e-mail, username or mobile number and show a message on the matching field.	Implemented
AUTH-3	The system shall refuse sign-up for users younger than 13.	Implemented
AUTH-4	The system shall store passwords only as bcrypt hashes.	Implemented
AUTH-5	The system shall e-mail a 6-digit OTP, store it only as a bcrypt hash, accept it for 5 minutes and 5 attempts, and allow a resend after 60 seconds.	Implemented
AUTH-6	The system shall keep an account unverified until the OTP is accepted.	Implemented
AUTH-7	The system shall let a user log in with e-mail or username and issue a JWT valid for 7 days.	Implemented
AUTH-8	The system shall let a user reset a forgotten password with an e-mail OTP.	Implemented
AUTH-9	The system shall ask for a fresh OTP when a user changes e-mail or mobile.	Implemented
AUTH-10	The system shall also deliver OTPs by SMS.	Planned
AUTH-11	The system shall allow only users aged 18 or over to enter or host paid tournaments.	Planned

3.2 Rooms and Matches

ID	Requirement	Status
ROOM-1	The system shall let a verified user create a room with name, format, capacity, languages, topics, questions, duration, visibility, optional per-question time and optional proctoring.	Implemented
ROOM-2	The system shall support Solo, Duo, Squad and Duel formats.	Implemented
ROOM-3	The system shall list public rooms in the Arena and let a user join a private room by invite code.	Implemented
ROOM-4	The system shall enforce room capacity and a minimum number of players before a host can start.	Implemented
ROOM-5	The system shall move a room through draft, open, live and ended states.	Implemented
ROOM-6	The system shall prevent a player leaving a proctored room once it is live.	Implemented
ROOM-7	The system shall provide a live scoreboard and fix final ranks when the host finalises the room.	Implemented

ID	Requirement	Status
ROOM-8	The system shall rank by score and break ties by the earlier last correct submission.	Implemented
ROOM-9	The system shall let a host eliminate a player and record the reason.	Implemented
ROOM-10	The system shall offer platform-run rooms and an always-open practice duel.	Implemented

3.3 Judging and Question Bank

ID	Requirement	Status
JDG-1	The system shall accept a submission naming room, question, language and code.	Implemented
JDG-2	The system shall reject a submission from a player who is not in the room, when the room is not live, or when the player is locked (HTTP 423).	Implemented
JDG-3	The system shall keep hidden test cases on the server and send only visible examples to the browser.	Implemented
JDG-4	The system shall execute JavaScript submissions against the hidden cases with a 2000 ms timeout.	Implemented
JDG-5	The system shall judge Java, Python, C++ and C by a deterministic simulation and mark the verdict simulated.	Simulated
JDG-6	The system shall award the question points plus a time bonus of 2 points per remaining minute, and count a question once per player.	Implemented
JDG-7	The system shall save a failed attempt as a mistake with the last code and failing case.	Implemented
JDG-8	The system shall allow practice on any question outside a room.	Implemented
JDG-9	The system shall run all languages in an isolated judge service.	Planned

3.4 Tournaments and Money

ID	Requirement	Status
MNY-1	The system shall keep a tournament as a draft until the host funds the prize pool.	Implemented
MNY-2	The system shall hold the funded pool and every entry fee in escrow and record each in the ledger.	Implemented
MNY-3	The system shall offer the presets winner takes all, top 3 (50/30/20), top 5 (40/25/15/10/10) and a custom split.	Implemented
MNY-4	The system shall deduct a platform fee equal to 1% of the prize pool, rounded to a whole paisa, before paying prizes.	Implemented
MNY-5	The system shall pay each ranked player according to the split and write a ledger row for each payment.	Implemented
MNY-6	The system shall block payout while a dispute on the room is open.	Implemented

ID	Requirement	Status
MNY-7	The system shall let the host or an administrator refund entries.	Implemented
MNY-8	The system shall store all amounts as integer paise and make the ledger append only.	Implemented
MNY-9	The system shall collect and pay out real money through a licensed payment gateway with verified payout accounts.	Planned

3.5 Proctoring

ID	Requirement	Status
PRC-1	The system shall require camera permission before a proctored match starts and record the camera state.	Implemented
PRC-2	The system shall accept webcam snapshots (up to 400 KB each) at an interval chosen by the host, at least 5 seconds.	Implemented
PRC-3	The system shall record violations of these types: tab switch, blur, exit full screen, context menu, copy, cut, paste, developer tools, other.	Implemented
PRC-4	The system shall lock a player on the server after any violation and notify the host.	Implemented
PRC-5	The system shall let the host view violations and snapshots and unlock or eliminate a player.	Implemented
PRC-6	The system shall keep snapshots in durable storage and delete them after 30 days.	Planned

3.6 Learning and Achievements

ID	Requirement	Status
LRN-1	The system shall show a player the mistakes list with last code, failing case and a retry action.	Implemented
LRN-2	The system shall provide a library of free books with the licence shown.	Implemented
LRN-3	The system shall compute seven achievements from stored statistics and show progress.	Implemented
LRN-4	The system shall offer an AI hint and an AI explanation for a failed attempt.	Implemented

3.7 Administration

ID	Requirement	Status
ADM-1	The system shall restrict administrator routes to users with the admin flag.	Implemented
ADM-2	The system shall let an administrator create, edit, retire and restore questions.	Implemented
ADM-3	The system shall let an administrator review and approve library books.	Implemented

ID	Requirement	Status
ADM-4	The system shall let an administrator list users and verify or promote them.	Implemented
ADM-5	The system shall let an administrator resolve disputes and view money totals and the ledger.	Implemented

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

The interface consists of fifteen pages that share one stylesheet set and a small script layer. The design uses a cream background with terracotta, amber and forest-green accents. On a phone, a bottom navigation bar replaces the top menu.

Page	Purpose
index.html	Landing page
signup.html, login.html, verify-otp.html	Account creation, login and OTP entry
arena.html	Browse and join public rooms
create-room.html, create-tournament.html	Create free rooms and paid tournaments
tournaments.html, tournament-details.html	List and details of paid contests
match.html, duel.html	Play a match or a duel
host-dashboard.html	Host's live view of contests
learning.html	Mistake book and library
profile.html	Profile, statistics and achievements
admin.html	Administrator back office

4.2 Software Interfaces

Interface	Use
MySQL-compatible database	Accessed through MySQL Connector/J (JDBC) with a HikariCP connection pool; TLS for TiDB Cloud
E-mail	Gmail SMTP, or the Brevo HTTPS API when a Brevo API key is configured
Groq API	Chat completions with model openai/gpt-oss-120b for hints and explanations
Payment gateway	Not connected; money is simulated

4.3 Communication Interfaces

The pages communicate with the server over HTTPS using JSON. Protected calls carry a bearer token in the Authorization header. Errors use standard status codes: 401 not signed in, 403 not allowed, 404 not found, 422 invalid form with a map of field messages, 423 player locked. A health endpoint at /api/health reports database, mail and AI status.

5. NON-FUNCTIONAL REQUIREMENTS

Area	Requirement
Security	Passwords and OTPs are stored only as bcrypt hashes. Sessions use signed JWTs checked on every protected request. SQL uses parameters. Hidden test cases are never sent to the browser. Admin routes check the flag on the server. Secrets are only in environment variables.
Integrity	Money uses integer paise. Settlement runs inside a database transaction. The ledger is append only. A solved question is scored once.
Performance	JavaScript runs are limited to 2000 ms. The server uses a pooled set of database connections. Scoreboard and room data are fetched by polling every few seconds.
Reliability	A proctoring lock is stored in the database and survives refresh and re-login. A health endpoint lets the host see whether the service and database are up.
Usability	Forms show field-level errors. Pages work at phone width. Empty states have written text. A new account shows zero in every counter.
Portability	The server runs wherever Java 17 and a MySQL-compatible database are available.
Maintainability	Routes, domain rules, judge, mail and AI access are separate modules; the judge sits behind one module so it can be replaced.
Privacy	Webcam snapshots are personal data. They are visible only to the host and administrators, and a retention limit is planned.

6. DATA REQUIREMENTS

6.1 Data Model

The database has twenty tables: users, otps, topics, languages, prize_presets, questions, books, achievements, ach_state, rooms, room_players, submissions, mistakes, violations, snapshots, payments, payouts, ledger, disputes and notifications.

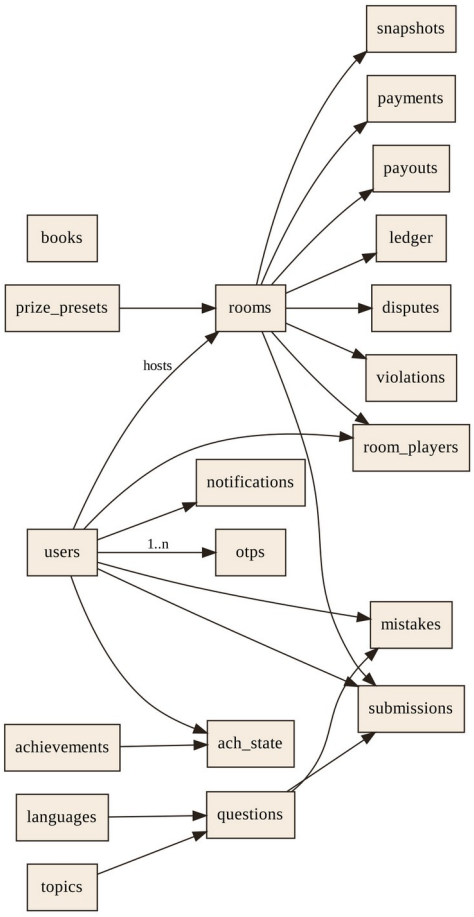


Figure 6.1: Entity relationship overview

6.2 Data Rules

- Identifiers are short prefixed strings, for example u_ for users and room_ for rooms.
- Timestamps are BIGINT milliseconds since the Unix epoch.
- E-mail, username and mobile are unique on users.
- A user can hold one seat per room (composite key on room_players).
- The ledger types are entry-fee, prize-fund, prize, refund and platform-fee.
- Seed data holds reference data only: 19 questions, 11 books, 7 achievements, 10 topics, 5 languages, 4 prize presets and 7 platform rooms.

7. SYSTEM MODELS

7.1 Sign-up and OTP Flow

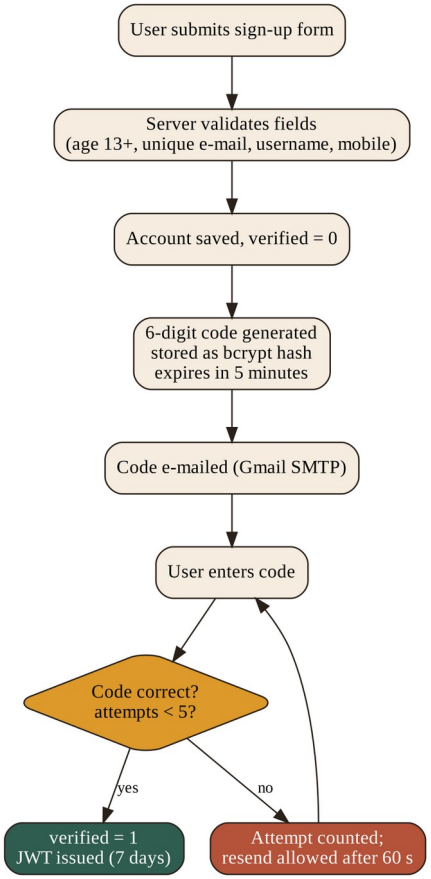


Figure 7.1: Sign-up and OTP verification

7.2 Match, Judging and Proctoring Flow

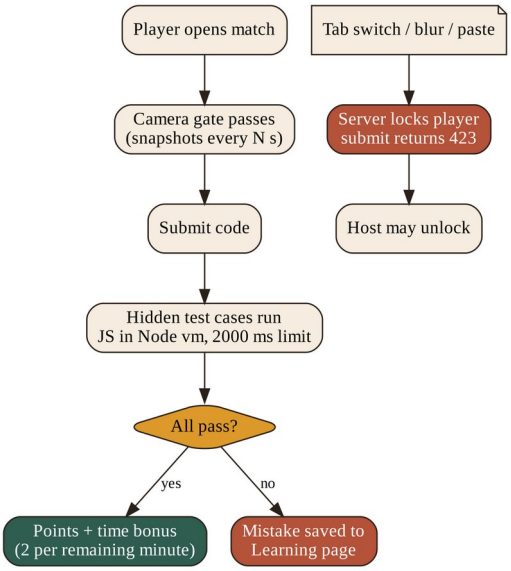


Figure 7.2: Match and proctoring flow

7.3 Tournament Money Flow

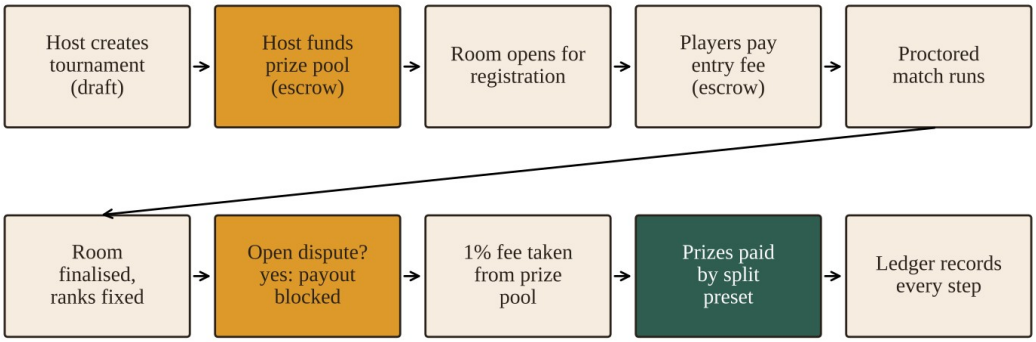


Figure 7.3: Tournament money flow

8. BUSINESS RULES AND REQUIREMENT STATUS

8.1 Business Rules

- BR-1: The platform fee is 1% of the prize pool, rounded to a whole paisa. It is not charged on entry fees.
- BR-2: Contests are skill-only. Ranking depends on solved questions, points and time.
- BR-3: A tournament cannot accept players until its prize pool is funded.
- BR-4: A dispute blocks payout until an administrator resolves it.
- BR-5: Any proctoring violation locks the player until the host unlocks them.
- BR-6: Platform counters start at zero for every new account.

8.2 Status Summary

Area	Status
Accounts, e-mail OTP, JWT sessions	Implemented
Rooms, duels, teams, scoreboard, ranks	Implemented
Hidden-test judging for JavaScript	Implemented
Judging for Java, Python, C++, C	Simulated; isolated judge planned
Escrow, 1% fee, presets, refunds, ledger	Implemented with a simulated gateway
Proctoring and host tools	Implemented; durable snapshot storage planned
Learning, achievements, AI coach	Implemented; AI needs an API key
Admin back office	Implemented
SMS OTP, real payments, 18+ rule	Planned

8.3 Future Requirements

- Integrate an isolated judge service such as Judge0.
- Integrate a licensed payment gateway with verified payout accounts.
- Add SMS OTP.
- Enforce the 18+ rule for paid contests.
- Store snapshots in object storage with a 30-day retention job.
- Add organisation hosting, a nightly ledger check and live scoreboards over WebSockets.

APPENDIX A: API ENDPOINTS

All paths are under /api. The server exposes 76 endpoints.

Route group	Endpoints
/auth (15)	POST signup, verify, resend, login, forgot, reset, password, payout, payout/verify, logout, me/contact, me/contact/confirm; GET otp/:id/meta, me; PATCH me
/rooms (12)	GET /, /:id, /code/:code, /:id/board; POST /, /:id/join, /:id/leave, /:id/start, /:id/eliminate, /:id/finalise, /:id/disputes; PATCH /:id
/play (6)	POST submit, practice; GET mistakes, submissions, achievements, notifications
/money (10)	POST rooms/:id/fund, entry, settle, refund; GET rooms/:id/summary, me/ledger, totals, ledger, payments, payouts
/proctor (8)	POST :roomId/violations, lock, unlock, camera, snapshots; GET :roomId/violations, snapshots, snapshots/:snapId
/ (content, 7)	GET bootstrap, questions, questions/:id, books, taxonomies, achievements; POST books
/admin (14)	GET questions, questions/:id, books, users, disputes, rooms/:id/proctoring; POST questions, questions/:id/retire, questions/:id/restore, books/:id/decide, users/:id/admin, users/:id/verify, disputes/:id/resolve; DELETE questions/:id
/ai (4)	POST hint, explain, coach; GET status

APPENDIX B: TRACEABILITY

Requirements	Module (owner)	Verification
AUTH-1 to AUTH-9	Accounts and OTP (Hiten Sharma)	Smoke test login; manual sign-up checks
ROOM-1 to ROOM-10	Rooms and matches (Hiten Sharma); Duels and teams (Arjun Suman)	Smoke test create, join, start, finalise
JDG-1 to JDG-8	Code judge and question bank (Harsh Sharma)	Smoke test: correct pass 4 of 4, wrong fails, simulated flag, hidden cases hidden
MNY-1 to MNY-8	Tournaments and money (Hemanth Singh Tanwar)	Smoke test: draft, fund, entry, settlement fee 500 paise, ledger
PRC-1 to PRC-5	Proctoring and host dashboard (Arjun Suman)	Proctor test: lock, HTTP 423, unlock
LRN-1 to LRN-4	Learning, achievements and AI (Akshat Rathore)	Mistake book checked; AI needs a key to test
ADM-1 to ADM-5	Admin and moderation (Harsh Sharma)	Smoke test admin gate and money totals
Platform and deployment	Platform core and DevOps (Hemanth Singh Tanwar); UI (Akshat Rathore)	Health endpoint, deployment on Render